

Penetration Test Report

Client: Acme Corp

Engagement Title: Web Application Penetration Test – OWASP Juice Shop

Prepared By: CyberSecure Solutions

Date: September 21, 2025

Version: 1.0

Table of Contents

1. Executive Summary
 2. Scope of Engagement
 3. Methodology
 4. Attacks Executed
 5. Detailed Findings
 6. Risk Ratings and Impact
 7. Recommendations
 8. Conclusion
 9. Appendix A – Tools Used
 10. Appendix B – References
-

1. Executive Summary

This penetration test was conducted against the OWASP Juice Shop application (<https://juice-shop.herokuapp.com/>).

The objective of this engagement was to evaluate the security posture of the target application by simulating real-world attack scenarios.

Overall Risk Posture: High

Key Findings:

- Critical SQL Injection vulnerabilities allowing authentication bypass and potential full system compromise.
- High severity injection attempts causing server errors that confirm exploitable injection flaws.
- Medium severity time-based injection attempts indicating potential denial of service risks.

The vulnerabilities identified, if exploited, could lead to compromise of sensitive data, unauthorized access, and reputational damage. Remediation steps are strongly recommended to improve the security of the environment.

2. Scope of Engagement

- **Target Application:** OWASP Juice Shop
 - **Testing Type:** Black Box
 - **Testing Window:** September 15, 2025 - September 21, 2025
 - **In-Scope Assets:**
 - <https://juice-shop.herokuapp.com/>
 - **Out-of-Scope Assets:** None specified
-

3. Methodology

Testing followed industry-standard frameworks and guidelines:

- OWASP Web Security Testing Guide (WSTG)
- NIST SP 800-115 Technical Guide to Information Security Testing
- PTES (Penetration Testing Execution Standard)

The engagement phases included:

1. Reconnaissance
 2. Threat Modeling
 3. Vulnerability Analysis
 4. Exploitation
 5. Post-Exploitation and Impact Analysis
 6. Reporting
-

4. Attacks Executed

The following attack types were executed against the target application:

- SQL Injection
-

5. Detailed Findings

Attack: SQL Injection

Successful Payloads

- ' OR 1=1 --
- ' OR '1'='1' --

Unsuccessful Payloads

- ' OR 1=0 --
- '
- ' UNION SELECT 1,2,3 FROM (SELECT COUNT(*),CONCAT(0x7c,VERSION()),0x7c,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a--
- ' || PG_SLEEP(5)--
- ' UNION SELECT null, username, password FROM users--

Risk Rating

- **Severity:** Critical / High
- **CVSS Vector:** Not explicitly provided; estimated 9.0+ for successful authentication bypass
- **Impact:** An attacker can bypass authentication, gain unauthorized admin access, potentially leading to full system compromise, data leakage, and denial of service.
- **Likelihood:** High

Evidence

- Successful authentication bypass using payloads such as ' OR 1=1 -- and ' OR '1'='1' -- returned valid authentication tokens with HTTP 200 status.
- Server errors (HTTP 500) with stack traces referencing Sequelize SQLite queries were observed with injection payloads, confirming vulnerability.
- Timing analysis showed no conclusive blind SQL injection delays but indicated injection attempts caused server instability.

Recommendation

Implement parameterized queries or prepared statements to prevent SQL injection. Validate and sanitize all user inputs rigorously. Employ ORM frameworks with built-in protections and ensure error handling does not leak sensitive information. Disable detailed error messages in production environments. Conduct thorough code reviews focusing on database query construction and input handling.

6. Risk Ratings and Impact

Finding	Severity	CVSS Score	Impact	Likelihood
SQL Injection	Critical / High	Estimated 9.0+	Full system compromise, unauthorized admin access, data leakage, denial of service	High

7. Recommendations

- **Short Term:** Patch identified SQL injection vulnerabilities immediately. Enforce strong input validation and use parameterized queries. Disable detailed error messages in production.
 - **Medium Term:** Implement secure software development lifecycle (SDLC) practices including code reviews and automated security scanning integrated into CI/CD pipelines.
 - **Long Term:** Conduct regular penetration testing and establish continuous monitoring and logging to detect and respond to suspicious activities promptly.
-

8. Conclusion

The assessment highlighted several critical vulnerabilities within the OWASP Juice Shop application, primarily SQL injection flaws that allow authentication bypass and potential full system compromise. Addressing these issues will significantly improve the overall security posture. Ongoing security testing and proactive monitoring are strongly recommended to prevent future risks and protect sensitive data.

9. Appendix A – Tools Used

- Burp Suite Pro
 - OWASP ZAP
 - Nmap
 - SQLMap
 - Nikto
 - Custom scripts for payload automation and timing analysis
-

10. Appendix B – References

- OWASP Top 10 (2021) – <https://owasp.org/Top10/>
 - NIST SP 800-115 – <https://csrc.nist.gov/publications/detail/sp/800-115/final>
 - PTES – <http://www.pentest-standard.org/>
-

End of Report